In [1]:

```
import argparse
import os
import time
import shutil
import contextlib

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.backends.cudnn as cudnn

import torchvision
import torchvision.transforms as transforms

import numpy as np

from models import *  # bring everything in the folder models

# ===================== Device & model ===================== #
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
print("=> Building model...")

batch_size = 128
model_name = "RESNET20"

# float=True: full-precision model
model = resnet20_cifar(float=True).to(device)

# ===================== Dataset & Dataloader ===================== #
normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447],
                                 std=[0.247, 0.243, 0.262])

train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ]))

trainloader = torch.utils.data.DataLoader(
    train_dataset, batch_size=batch_size,
    shuffle=True, num_workers=2
)

test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
```

```
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))

testloader = torch.utils.data.DataLoader(
    test_dataset, batch_size=batch_size,
    shuffle=False, num_workers=2
)

print_freq = 100  # every 100 batches, accuracy printed


# ===================== Utility classes & functions ===================== #
class AverageMeter(object):
    """Computes and stores the average and current value"""

    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count


def accuracy(output, target, topk=(1, 5)):
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk)
    batch_size = target.size(0)

    _, pred = output.topk(maxk, 1, True, True)  # values, indices
    pred = pred.t()
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].reshape(-1).float().sum(0)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res


def save_checkpoint(state, is_best, fdir):
    filepath = os.path.join(fdir, 'checkpoint.pth')
    torch.save(state, filepath)
    if is_best:
        shutil.copyfile(filepath, os.path.join(fdir, 'model_best.pth.tar'))
```

```python
def adjust_learning_rate(optimizer, epoch):
    """For resnet, the lr starts from 0.1, and is divided by 10 at 80 and 12
    adjust_list = [80, 120]
    if epoch in adjust_list:
        for param_group in optimizer.param_groups:
            param_group['lr'] = param_group['lr'] * 0.1


# ===================== Train & Validate ===================== #
def train(trainloader, model, criterion, optimizer, epoch):
    batch_time = AverageMeter()
    data_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    model.train()
    end = time.time()

    for i, (inp, target) in enumerate(trainloader):
        data_time.update(time.time() - end)

        inp, target = inp.to(device), target.to(device)

        output = model(inp)
        loss = criterion(output, target)

        prec = accuracy(output, target)[0]
        losses.update(loss.item(), inp.size(0))
        top1.update(prec.item(), inp.size(0))

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        batch_time.update(time.time() - end)
        end = time.time()

        if i % print_freq == 0:
            print('Epoch: [{0}][{1}/{2}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Data {data_time.val:.3f} ({data_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                epoch, i, len(trainloader),
                batch_time=batch_time,
                data_time=data_time,
                loss=losses,
                top1=top1))


def validate(val_loader, model, criterion):
    batch_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    model.eval()
```

```
    end = time.time()

    with torch.no_grad():
        for i, (inp, target) in enumerate(val_loader):
            inp, target = inp.to(device), target.to(device)

            output = model(inp)
            loss = criterion(output, target)

            prec = accuracy(output, target)[0]
            losses.update(loss.item(), inp.size(0))
            top1.update(prec.item(), inp.size(0))

            batch_time.update(time.time() - end)
            end = time.time()

            if i % print_freq == 0:
                print('Test: [{0}/{1}]\t'
                      'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                      'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                      'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                    i, len(val_loader),
                    batch_time=batch_time,
                    loss=losses,
                    top1=top1))

    print(' * Prec {top1.avg:.3f}% '.format(top1=top1))
    return top1.avg


# ====================== (Optional) Training block ====================== #
# 如果你已经训练过并且有 model_best.pth.tar, 可以直接跳过这段
"""
lr = 0.1
weight_decay = 5e-4
epochs = 150
best_prec = 0

model = model.to(device)
criterion = nn.CrossEntropyLoss().to(device)
optimizer = torch.optim.SGD(model.parameters(),
                            lr=lr,
                            momentum=0.9,
                            weight_decay=weight_decay)

if not os.path.exists('result'):
    os.makedirs('result')

fdir = 'result/' + str(model_name)
if not os.path.exists(fdir):
    os.makedirs(fdir)

for epoch in range(0, epochs):
    adjust_learning_rate(optimizer, epoch)
    train(trainloader, model, criterion, optimizer, epoch)
```

```
    print("Validation starts")
    prec = validate(testloader, model, criterion)

    is_best = prec > best_prec
    best_prec = max(prec, best_prec)
    print('best acc: {:1f}'.format(best_prec))

    save_checkpoint({
        'epoch': epoch + 1,
        'state_dict': model.state_dict(),
        'best_prec': best_prec,
        'optimizer': optimizer.state_dict(),
    }, is_best, fdir)
"""

# ====================== Load best checkpoint & eval once ==================
if not os.path.exists('result'):
    os.makedirs('result')
fdir = os.path.join('result', str(model_name))

best_path = os.path.join(fdir, 'model_best.pth.tar')
print("Loading checkpoint:", best_path)
checkpoint = torch.load(best_path, map_location=device)
model.load_state_dict(checkpoint['state_dict'])

criterion = nn.CrossEntropyLoss().to(device)
model.eval()
model.to(device)

print("Eval best checkpoint on test set:")
prec = validate(testloader, model, criterion)


# ======================  manual block check  ====================== #
class SaveInput:
    def __init__(self):
        self.inputs = []

    def __call__(self, module, module_in):
        self.inputs.append(module_in)

    def clear(self):
        self.inputs = []


save_input0 = SaveInput()
save_input1 = SaveInput()

block0 = model.layer1[0]
block1 = model.layer1[1]

block0.register_forward_pre_hook(save_input0)
block1.register_forward_pre_hook(save_input1)

dataiter = iter(trainloader)
images, labels = next(dataiter)
```

```
single_image = images[0].unsqueeze(0).to(device)  # shape [1, 3, 32, 32]

_ = model(single_image)

x0 = save_input0.inputs[0][0]  # input of BasicBlock0, shape [1, C_in, H, W]
input_b1 = save_input1.inputs[0][0]  # input of BasicBlock1 (output of Basic

# Manual recompute block0
y1 = F.conv2d(x0, block0.conv1.weight, bias=block0.conv1.bias,
              stride=block0.conv1.stride, padding=1)
y1 = F.batch_norm(y1,
                  block0.bn1.running_mean,
                  block0.bn1.running_var,
                  block0.bn1.weight,
                  block0.bn1.bias,
                  training=False,
                  eps=block0.bn1.eps)
y1 = F.relu(y1)

y2 = F.conv2d(y1, block0.conv2.weight, bias=block0.conv2.bias,
              stride=block0.conv2.stride, padding=1)
y2 = F.batch_norm(y2,
                  block0.bn2.running_mean,
                  block0.bn2.running_var,
                  block0.bn2.weight,
                  block0.bn2.bias,
                  training=False,
                  eps=block0.bn2.eps)

if block0.downsample is not None:
    res = block0.downsample(x0)
else:
    res = x0

out_b0 = F.relu(y2 + res)

diff = torch.norm(out_b0 - input_b1)
print("Difference between manual and hooked Block1.conv1 input:", diff.item(


# ==================== Hadamard vs baseline (per-layer MSE) ===============
def quantize_dequant_tensor(x: torch.Tensor, bits: int,
                            per_channel: bool = False, ch_axis: int = 0):
    """
    Symmetric quantization + dequantization:
    range: [-2^{b-1}+1, 2^{b-1}-1]
    """
    qmin = -(2 ** (bits - 1)) + 1
    qmax = (2 ** (bits - 1)) - 1

    if per_channel:
        dims = tuple(d for d in range(x.ndim) if d != ch_axis)
        max_abs = x.detach().abs().amax(dim=dims, keepdim=True).clamp(min=1e
        scale = max_abs / qmax
    else:
        max_abs = x.detach().abs().max().clamp(min=1e-12)
```

```
        scale = max_abs / qmax

    x_int = torch.round(x / scale).clamp(qmin, qmax)
    x_hat = x_int * scale
    return x_hat


def hadamard_matrix(n, device=None, dtype=torch.float32):
    """
    Generate n×n Hadamard matrix (n must be power of 2).
    Unnormalized version: H^{-1} = H^T / n.
    """
    assert (n & (n - 1)) == 0, "n must be a power of 2"
    H = torch.tensor([[1.0]], device=device, dtype=dtype)
    while H.size(0) < n:
        H = torch.cat([
            torch.cat([H, H], dim=1),
            torch.cat([H, -H], dim=1)
        ], dim=0)
    return H

def compare_hadamard_vs_baseline_on_conv(conv_layer: nn.Conv2d,
                                         x_in: torch.Tensor,
                                         bits: int = 4):
    """
    在指定的一层 conv 上比较:
      - baseline: 直接对 x, W 量化 -> y_base
      - hadamard: 对 x 做 x' = H x, 对 W 做 W' = W H^{-1}, 在 x', W' 上量化 -> y
    然后分别和 full-precision y_ref 比较 MSE。
    """

    conv = conv_layer
    x    = x_in.detach().to(device)          # [1, C_in, H, W]
    w    = conv.weight.detach().to(device)   # [C_out, C_in, kH, kW]
    bias = conv.bias.detach().to(device) if conv.bias is not None else None

    # 1) full-precision reference
    y_ref = F.conv2d(x, w, bias=bias,
                     stride=conv.stride,
                     padding=conv.padding)

    # 2) baseline quantization in original domain
    x_q  = quantize_dequant_tensor(x, bits=bits, per_channel=False)
    w_q  = quantize_dequant_tensor(w, bits=bits,
                                   per_channel=True, ch_axis=0)
    y_base = F.conv2d(x_q, w_q, bias=bias,
                      stride=conv.stride,
                      padding=conv.padding)

    mse_base = (y_base - y_ref).pow(2).mean().item()

    # 3) Hadamard transform domain quantization
    C_in = w.shape[1]
    assert (C_in & (C_in - 1)) == 0, \
        f"in_channels={C_in} must be power of 2 for Hadamard"
```

```
    # H: [C_in, C_in],  H^{-1} = H^T / C_in
    H     = hadamard_matrix(C_in, device=device, dtype=x.dtype)
    H_inv = H.t() / float(C_in)

    # (a) Activation transform: x' = H x
    # x: [B, C_in, H, W]
    # H: [C_in, C_in]
    # x_h[b, j, h, w] = sum_c H[j, c] * x[b, c, h, w]
    x_h = torch.einsum('jc,bchw->bjhw', H, x)

    # (b) Weight transform: W' = W H^{-1}
    # w: [C_out, C_in, kH, kW] -> indices: o i k l
    # H_inv: [C_in, C_in]      -> indices: i j
    # w_h[o, j, k, l] = sum_i w[o, i, k, l] * H_inv[i, j]
    w_h = torch.einsum('oikl,ij->ojkl', w, H_inv)

    # (c) Quantize in Hadamard domain
    x_h_q = quantize_dequant_tensor(x_h, bits=bits, per_channel=False)
    w_h_q = quantize_dequant_tensor(w_h, bits=bits,
                                    per_channel=True, ch_axis=0)

    # (d) Convolution with transformed & quantized weights/activations
    y_h = F.conv2d(x_h_q, w_h_q, bias=bias,
                   stride=conv.stride,
                   padding=conv.padding)

    mse_h = (y_h - y_ref).pow(2).mean().item()

    print("========== Hadamard vs Baseline on one Conv layer ==========")
    print(f"Layer: {conv}")
    print(f"{bits}-bit baseline MSE       : {mse_base:.6e}")
    print(f"{bits}-bit Hadamard-domain MSE: {mse_h:.6e}")
    if mse_base > 0:
        print(f"MSE ratio (Had / base)      : {mse_h / mse_base:.3f}")
    print("============================================================")

compare_hadamard_vs_baseline_on_conv(block0.conv1, x0, bits=4)
compare_hadamard_vs_baseline_on_conv(block0.conv2, x0, bits=4)
```

```
Using device: cpu
=> Building model...
Loading checkpoint: result/RESNET20/model_best.pth.tar
Eval best checkpoint on test set:
Test: [0/79]    Time 2.479 (2.479)      Loss 0.2619 (0.2619)    Prec 91.406%
(91.406%)
 * Prec 92.500%
Difference between manual and hooked Block1.conv1 input: 0.0
========== Hadamard vs Baseline on one Conv layer ==========
Layer: Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
4-bit baseline MSE       : 5.339587e-03
4-bit Hadamard-domain MSE: 4.356371e-03
MSE ratio (Had / base)      : 0.816
============================================================
========== Hadamard vs Baseline on one Conv layer ==========
Layer: Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
4-bit baseline MSE       : 2.890424e-03
4-bit Hadamard-domain MSE: 3.107736e-03
MSE ratio (Had / base)      : 1.075
============================================================
```

In [ ]: